



JavaScript Classes & Object-Oriented Programming

Concepts • Classes • Methods • Inheritance • Composition

Today's Agenda



The Mental Model

Class vs instance, properties vs methods — concepts before code



From Object to Class

Object literals → constructors → the new keyword



Methods & this

Behaviors, this keyword, and Promises in class methods



Design Challenge

Design classes for a food delivery app



Inheritance & Composition

extends, super(), has-a vs is-a, when to use which



The Mental Model

Concepts before code — understanding classes intuitively

Class vs. Instance

Class = The Concept

"Car"

A class is an idea — a blueprint.
It exists in your mind.

It describes what something is and
what it can do, but it's not a real thing.

Instance = The Real Thing



That red Tesla parked outside.
Your neighbor's blue Honda.

An instance is a specific, real thing
created from the blueprint.

One class → many instances. Each instance has its own data.

Properties, Methods & Static



Properties

Characteristics of a specific car

`color, brand, speed, fuelLevel, mileage`

What it IS — the data that makes each instance unique.



Methods

Capabilities — things the car can do

`drive(), honk(), refuel(), brake()`

What it DOES — behaviors that use or change its properties.



Static

Belong to the concept, not any one car

`Car.numberOfWheels, Car.compare(a, b)`

Shared truths — same for ALL cars, not specific to any instance.

The Full Picture

CLASS: Car

— The Blueprint —

Properties:

brand, color, speed, fuel

Methods:

drive(), honk(), refuel()

Static:

numberOfWheels = 4



new

Instance: myCar

brand: "Tesla"
color: "red"
speed: 0

`myCar.drive()` ✓

Instance: yourCar

brand: "Honda"
color: "blue"
speed: 60

`yourCar.drive()` ✓



From Object to Class

Translating the mental model into JavaScript

Starting Point: An Object

You already know how to describe a car in JavaScript:

```
const myCar = {  
  brand: "Tesla"  
  color: "red"  
  speed: 0  
  
  drive() {  
    console.log("Vroom!");  
  }  
};
```

This works fine for ONE car. But what if you need 50?

Copy-pasting 50 objects is a nightmare. You need a blueprint (describe data + behavior) → a class.

Your First Class

```
class Car {  
  
  constructor(brand, color) {  
    this.brand = brand;  
    this.color = color;  
    this.speed = 0;  
  }  
}
```

class keyword
defines the blueprint

constructor runs
automatically when
you call new

this.____ assigns
properties to the
new instance

```
const myCar = new Car("Tesla", "red");  
const yourCar = new Car("Honda", "blue");
```

What does `new` actually do?

1

Creates an empty object

JavaScript creates a brand new, empty `{}` behind the scenes.

2

Sets `this` to that object

Inside the constructor, `this` now refers to the new empty object.

3

Runs the constructor

Your constructor code runs: `this.brand = brand` assigns properties.

4

Returns the object

The finished object (now with all your properties) is returned to you.



Exercise: Your First Class

1. Create a class called **Student** with properties:

`name, age, grade`

2. Create 3 instances with different values.

3. `Console.log` each student's name and grade.

Bonus: Add a graduated property that defaults to false.



Methods & this

Giving your classes behavior

Adding Methods to a Class

```
class Car {  
  constructor(brand, color) {  
    this.brand = brand;  this.color = color;  this.speed = 0;  
  }  
  
  drive(kmh) {  
    this.speed = kmh;  
    console.log(`${this.brand} driving at ${kmh} km/h`);  
  }  
  
  brake() {  
    this.speed = 0;  
  }  
}
```

Methods are just functions defined inside the class. No function keyword needed.

this = The Thing Left of the Dot

That's really all you need to know. When you call a method, this is the object you called it on.

```
const tesla = new Car("Tesla", "red");  
const honda = new Car("Honda", "blue");  
  
tesla.drive(100); // this = tesla → "Tesla driving at 100 km/h"  
honda.drive(60); // this = honda → "Honda driving at 60 km/h"
```

tesla . drive(100)

↑
this is tesla

Same method, different instance → different this → different result.

Methods Calling Other Methods

Methods can use this to call other methods on the same instance:

```
class Car {  
  // ...constructor...  
  
  drive(kmh) { this.speed = kmh; }  
  brake()    { this.speed = 0; }  
  
  emergencyStop() {  
    console.log("Emergency! Braking now.");  
    this.brake();  
    console.log(`Speed is now ${this.speed}`);  
  }  
}
```

this.brake() calls the brake method on the same car instance. Methods can compose together.

Async Methods & Promises

Class methods can be async — just add the `async` keyword. Works exactly like regular async functions.

```
class Car {  
  // ...constructor, drive, brake...  
  
  async fetchMaintenanceHistory() {  
    const response = await fetch(`/api/cars/${this.brand}/history`);  
    const data = await response.json();  
    return data;  
  }  
}  
  
// Usage:  
const history = await myCar.fetchMaintenanceHistory();
```

async/await inside a method works the same way. The method returns a Promise.

Static Methods & Properties

Belong to the class itself, not to any instance. Called on the class name directly.

```
class Car {  
    static numberOfWheels = 4  
;  
  
    static compare(carA, carB) {  
        return carA.speed - carB.speed;  
    }  
}  
  
Car.numberOfWheels;           // 4 – called on the class  
Car.compare(myCar, yourCar); // compare two instances
```

✓ `Car.numberOfWheels`

✗ `myCar.numberOfWheels`

Static = called on the class name. Regular = called on an instance.



Exercise: Add Methods

Using your Student class from before:

1. Add a **study(subject)** method that logs the student's name + subject.
2. Add a **graduate()** method that sets graduated to true.
3. Add a **static compare(a, b)** that returns the difference in grades.

Bonus: Add an `async fetchTranscript()` that simulates an API call.



Design Challenge

Design classes for a food delivery app

Rules:

1. Paper only — no code yet!
2. For each class: name, properties, methods
3. Show relationships between classes

The Brief: FoodDash App

You're building a food delivery app. Customers browse restaurants, add items to their order, and a driver picks it up and delivers it. Design the classes that make this work.

Think about:

- 1 What classes do you need?
- 2 What properties does each class have?
- 3 What methods does each class need?
- 4 How do the classes relate to each other?
- 5 Does anything share behavior? How would you handle that?



8 minutes — then each team presents their design!

Let's Compare Designs

Which classes did you create? Did different teams pick different ones?

Did anyone make Driver extend User? Customer extend User?

How did you handle the relationship between Order and Restaurant?

Did any team give Order a method that calls methods on other objects?

Where did shared behavior come up? How did you handle it?

That last question leads us to the next topic...



Inheritance & Composition

Two ways to share behavior — and when to use which

Inheritance: extends & super()

A child class inherits all properties and methods from a parent class.

```
class Vehicle {
  constructor(brand, speed) {
    this.brand = brand; this.speed = speed;
  }
  move() { console.log(`${this.brand} is moving`); }
}

class Car extends Vehicle {
  constructor(brand, speed, doors) {
    super(brand, speed); // calls Vehicle's constructor
    this.doors = doors;
  }
  honk() { console.log("Beep!"); } // Car has move() from Vehicle + its own honk()
}
```

extends = "is a". A Car IS A Vehicle. It gets everything Vehicle has, plus its own stuff.

When Inheritance Works



Clean "is-a" relationship

```
Vehicle → Car, Truck, Motorcycle  
Shape → Circle, Rectangle, Triangle  
Account → CheckingAccount, SavingsAccount
```

It works when:

- ✓ The child truly IS A type of the parent (not just shares some behavior).
- ✓ The parent is stable and unlikely to change in ways that break children.
- ✓ You need only one level of inheritance (avoid deep chains).



When Inheritance Gets Awkward

```
class Vehicle {  
  refuel() { console.log("Filling up the tank..."); }  
}  
  
// ⚡ Uh oh...  
class ElectricCar extends Vehicle {  
  refuel() { throw new Error("I don't use fuel!"); }  
}
```

The problem:

- ✗ ElectricCar IS A Vehicle, but can't refuel. Inheritance forces behavior that doesn't fit.
- ✗ Adding charge() to Vehicle would break for gas cars that don't charge.
- ✗ You end up overriding methods just to disable them — a code smell.

Composition: "Has-A" Instead of "Is-A"

Instead of inheriting behavior, a Car HAS parts. Each part is its own class.

```
class Engine {  
  start() { console.log("Engine started"); }  
}  
  
class GPS {  
  navigate(to) { console.log(`Navigating to ${to}`); }  
}  
  
class Car {  
  constructor(brand) {  
    this.brand = brand;  
    this.engine = new Engine();  
    this.gps = new GPS();  
  }  
  start() { this.engine.start(); }  
  goTo(address) { this.gps.navigate(address); }  
}
```

Car doesn't inherit Engine — it creates one and delegates to it. Simple and clear.

Next Level: Passing Dependencies In

What if you want different engines? Instead of creating one inside, pass it from outside.

```
// Different engines — same interface (both have a start method)
class ElectricEngine {
  start(brand) { console.log(`${brand}: electric engine humming`); }
}

class GasEngine {
  start(brand) { console.log(`${brand}: gas engine roaring`); }
}

// Car receives its engine from outside — doesn't create it
class Car {
  constructor(brand, engine) {
    this.brand = brand; this.engine = engine;
  }
  start() { this.engine.start(this.brand); }
}

new Car("Tesla", new ElectricEngine()).start(); // "Tesla: electric engine humming"
new Car("Ford", new GasEngine()).start();      // "Ford: gas engine roaring"
```

Key: both engines have a `start()` method. The `Car` only cares about that — not which type it is.

Back to FoodDash: Inheritance Approach

```
class User { constructor(name, email) { ... } }  
  
class Customer extends User {  
  placeOrder() { ... }  
}  
  
class Driver extends User {  
  deliver() { ... }  
}  
  
// 🤔 But what if a driver is ALSO a customer?  
// JS doesn't support multiple inheritance!
```

Inheritance forces you into a single hierarchy. Real-world roles don't always fit neatly.

FoodDash: Composition Approach

```
// Both roles have the same method: perform()
const customerRole = {
  perform(name) { console.log(`${name} is placing an order`); }
};
const driverRole = {
  perform(name) { console.log(`${name} is delivering an order`); }
};

// User delegates to its role – doesn't care which one
class User {
  constructor(name, email, role) {
    this.name = name; this.email = email; this.role = role;
  }
  doWork() { this.role.perform(this.name); }
}

new User("Alex", "a@m.com", customerRole).doWork();
new User("Sam", "s@m.com", driverRole).doWork();
```

Notice the contract: both roles must have a `perform()` method. In TypeScript, you'd enforce this with an **interface**. In JS, it's a convention — document it, and your team follows it.

Inheritance vs. Composition

	Inheritance (is-a)	Composition (has-a)
Relationship	"A Car IS A Vehicle"	"A Car HAS an Engine"
Coupling	Tight — child depends on parent	Loose — objects are independent
Flexibility	Hard to change later	Easy to swap behaviors
Reusability	Limited to the hierarchy	Mix & match freely
Complexity	Simple for 1 level	Slightly more setup
Best for	Clear type hierarchies	Shared behaviors across unrelated classes

Rule of thumb: Favor composition. Use inheritance only when there's a clear, stable "is-a" relationship.

Optional / Bonus



Common Design Patterns

Named solutions to problems you'll see over and over again

What Are Design Patterns?

Reusable solutions to common problems in software design. Not code you copy-paste — they're templates for how to structure your classes and objects.

Where do they come from? The concept was popularized by the "Gang of Four" book *Design Patterns: Elements of Reusable Object-Oriented Software* (1994). It catalogued 23 patterns that keep showing up across languages and codebases.

Why should you care?

- Shared vocabulary — say "Factory" and your whole team knows the shape of the solution
- Proven approaches — these aren't invented, they're discovered from real codebases
- Easier refactoring — patterns make code more flexible and easier to change later
- Faster onboarding — new devs recognize patterns and understand intent quickly

Caveat: Don't force patterns where they aren't needed. If your code is simple and clear, leave it alone.

Strategy Pattern

You already know this one! Swap behavior by passing in a different object.

```
// The "strategy" is the engine – swappable at construction time
const electric = { start(b) { console.log(`${b}: humming`); } };
const gas      = { start(b) { console.log(`${b}: roaring`); } };
const hybrid   = { start(b) { console.log(`${b}: both!`); } };

// Same class, different strategy → different behavior
new Car("Tesla", electric).start();
new Car("Ford",   gas).start();
new Car("Toyota", hybrid).start();
```

When to use it:

- You have multiple ways to do the same thing (sorting, validation, pricing, auth...)
- You want to switch behavior without modifying the class
- You want to test different behaviors in isolation

Factory Pattern

A function that creates objects for you — hides the new keyword and setup logic.

```
function createCar(type, brand) {  
  const engines = {  
    electric: { start(b) { console.log(`${b}: humming`); } },  
    gas:      { start(b) { console.log(`${b}: roaring`); } },  
  };  
  return new Car(brand, engines[type]);  
}  
  
// Caller doesn't need to know about engines or how Car is built  
const tesla = createCar("electric", "Tesla");  
const ford  = createCar("gas", "Ford");
```

When to use it:

- Object creation is complex (many params, config, dependencies)
- You want to centralize and hide construction details

Observer Pattern

"When something happens, notify everyone who cares." Think `addEventListener`.

```
class Order {  
  constructor() { this.listeners = []; this.status = "pending"; }  
  
  onChange(fn) { this.listeners.push(fn); }  
  
  updateStatus(newStatus) {  
    this.status = newStatus;  
    this.listeners.forEach(fn => fn(this.status));  
  }  
}  
  
const order = new Order();  
order.onChange(s => console.log(`Customer notified: ${s}`));  
order.onChange(s => console.log(`Driver notified: ${s}`));  
order.updateStatus("ready"); // both callbacks fire!
```

This is how DOM events, Node EventEmitter, and most UI frameworks work under the hood.

Singleton Pattern

Only one instance ever exists. Every call returns the same object.

```
class Database {  
  constructor(url) {  
    if (Database.instance) return Database.instance;  
    this.url = url;  
    this.connected = false;  
    Database.instance = this;  
  }  
  connect() { this.connected = true; }  
}  
const db1 = new Database("postgres://...");  
const db2 = new Database("mysql://...");  
console.log(db1 === db2); // true - same instance!
```

Good for:

DB connections, config, logging,
caches — things you truly need
only one of.

Use sparingly:

Singletons are global state in
disguise. They make testing harder
and hide dependencies.



Bonus Exercise: Build FoodDash

Now code it! Using your design from the challenge:

1. Create a Restaurant class with a menu (array of items).
2. Create an Order class that takes items + a restaurant.
3. Add an async calculateTotal() method to Order.
4. Create a User class that receives a role via the constructor.

Bonus: Add a static Order.sortByTotal(orders) method.